

Машина з кульками

Мадіна винайшла машину з кульками, щоб розважати учасників IOI. Внутрішня структура машини така:

- Машина складається з N **вершин**, індексованих від 0 до $N - 1$. Вершина $N - 1$ називається **коренем**.
- Для кожної вершини u з $0 \leq u < N - 1$, **батьківською** вершиною для u є вершина $P[u]$, де $P[u] > u$, а вершина u є **дочірною** для вершини $P[u]$. Корінь не має батьківської вершини. Вершина без дочірніх вершин називається **листом**.

Ви **не** знаєте N або $P[u]$ будь-якої вершини u . Натомість Мадіна повідомляє вам M , кількість листків та те, що індекси листків — це $0, 1, \dots, M - 1$. Ваше завдання — визначити внутрішню структуру машини, керуючи нею.

Кожна вершина машини може містити не більше одну **кульку**, і кожна кулька має **значення**, яке є невід'ємним цілим числом. Ми кажемо, що вершина має значення x , якщо вона містить кульку зі значенням x . Вершина вважається **зайнятою**, якщо вона містить кульку; інакше вона є **вільною**. Спочатку кожна вершина є вільною.

Ви можете виконувати два види операцій:

- **insert** (U, X) : спроба вставити нову кульку зі значенням X на листок U .
 - Якщо листок U зайнятий, операція повертає `false` без вставки кульки.
 - Якщо листок U вільний, кулька розміщується у вершині U . Потім кулька багаторазово рухається до батьківської вершини своєї поточної вершини, поки ця батьківська вершина є вільною. Рух зупиняється, коли кулька досягає кореня або вершини, батьківська вершина якої зайнята. Операція повертає `true`.
- **collect**(): якщо корінь вільний (що означає, що машина порожня), `collect` повертає порожній масив. В іншому випадку, рекурсивна процедура `traverse` описана нижче, збирає значення всіх кульок, які наразі знаходяться в машині, в масив S . Спочатку масив S порожній, і викликається `traverse(N - 1)`.

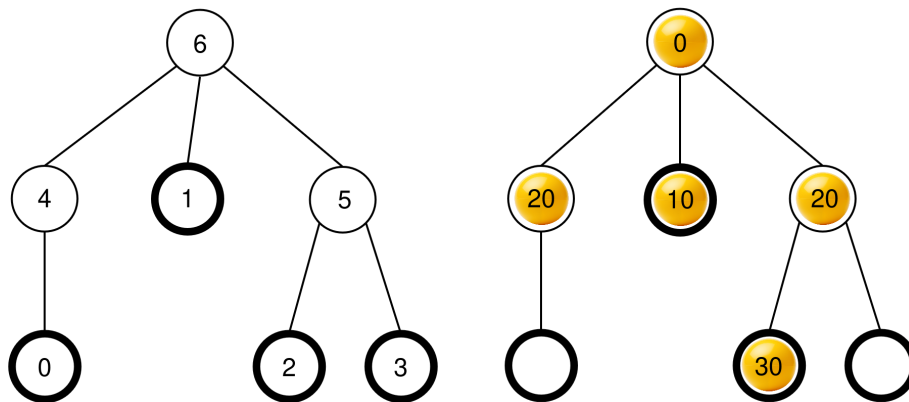
```

traverse(u) :
    додати значення кульки у вершині u у кінець S
    нехай c[u] буде списком зайнятих дітей u
    сортувати c[u] у неспадаючому порядку відповідно до значень кульок у
        цих вершинах (якщо кілька вершин мають однакове значення,
        вони можуть з'являтися в будь-якому порядку)
    для кожної вершини v в c[u]:
        traverse(v)

```

Після завершення цієї процедури **всі кульки видаляються** з машини, а `collect` повертає S .

Наприклад, розглянемо машину з $N = 7$ вершинами та батьківським масивом $P = [4, 6, 5, 5, 6, 6]$. На зображенні ліворуч показано порожню машину з індексами вершин, тоді як на зображенні праворуч показано можливу конфігурацію кульок після деякої послідовності операцій `insert` (ця послідовність наведена в розділі Приклад).



Коли викликається `collect()`, корінь (вершина 6) зайнятий, тому S ініціалізується як $S = []$ та викликається `traverse(6)`.

traverse(6) : вершина 6 має значення 0; додавання її до S дає $S = [0]$. Дочірніми вершинами 6 є 4, 1 та 5, всі з яких зайняті. Їхні значення 20, 10 та 20 відповідно. Сортування у неспадаючому порядку дає $c[6] = [1, 5, 4]$ (зауважте, що $c[6] = [1, 4, 5]$ також буде коректним, оскільки вершини 4 та 5 мають однакове значення). Потім процедура викликає `traverse` для кожної вершини в $c[6]$ у цьому порядку.

- **traverse(1)** : вершина 1 має значення 10; додавання її до S дає $S = [0, 10]$. Вершина 1 не має зайнятих дочірніх вершин, тому цей виклик завершується.
- **traverse(5)** : вершина 5 має значення 20; додавання її до S дає $S = [0, 10, 20]$. Вершина 5 має одну зайняту дочірню вершину, вершину 2, тому $c[5] = [2]$ і процедура викликає `traverse(2)`.
 - **traverse(2)** : вершина 2 має значення 30; додавання її до S дає $S = [0, 10, 20, 30]$. Вершина 2 не має зайнятих дочірніх вершин, тому цей виклик завершується.
 - Виконання повертається до `traverse(5)`, який обробив усі дочірні вершини в $c[5]$, тому його виклик завершується.

- **traverse(4)** : вершина 4 має значення 20; додавання її до S дає $S = [0, 10, 20, 30, 20]$.
Вершина 4 не має зайнятих дочірніх вершин, тому цей виклик завершується.

Усі виклики завершено, і більше немає вершин для обробки. Зрештою, кожна кулька виймається з машини, і `collect` повертає масив $S = [0, 10, 20, 30, 20]$.

Ваше завдання — визначити кількість вершин N та повідомити масив батьків $R = [R[0], R[1], \dots, R[N-2]]$, який описує структуру машини, але з потенційно різною міткою вершин, які не є листям і не коренем. Формально, повідомлений масив R вважається коректним, якщо кожній вершині u машини можна призначити унікальну мітку $L[u]$ з $0 \leq L[u] < N$ таким чином, що:

- $L[u] = u$ для всіх $0 \leq u < M$ та $u = N-1$, та
- $L[P[u]] = R[L[u]]$ для всіх $0 \leq u < N-1$.

Не обов'язково, щоб $R[u] > u$. Машина **повинна бути порожньою**, коли ви повідомляєте своє рішення.

Нехай K — кількість виконаних операцій `collect`, а B — максимальне значення, записане на будь-якій кулі після всіх викликів `insert`. Нехай $C = K + B$. Тоді C **не повинен перевищувати 1000**. Ваш бал у деяких підзадачах залежить від значення C .

Деталі реалізації

Вам слід реалізувати наступну процедуру.

```
std::vector<int> find_structure(int M)
```

- M : кількість листків у машині.
- Процедура викликається рівно один раз для кожного тестового випадку.
- Процедура повинна повертати масив $R = [R[0], R[1], \dots, R[N-2]]$ довжиною $N-1$, коректний масив батьківських елементів.

Для взаємодії з машиною ваша процедура може викликати наступні дві процедури.

Перша процедура така:

```
bool insert(int U, int X)
```

- U : індекс листка, куди слід вставити кульку. Повинна бути виконана умова $0 \leq U < M$.
- X : ціле значення кульки. Повинна бути виконана умова $0 \leq X \leq 1000$.
- Ця процедура повертає `true` якщо куля була успішно вставлена, або `false` якщо листок U вже був зайнятий.
- Цю процедуру можна викликати максимум 500 000 разів.

Друга процедура така:

```
std::vector<int> collect()
```

- Збирає значення для всіх вставлених кульок, очищує машину та повертає зібраний масив S .

Якщо в будь-який момент під час виконання вашої програми значення C перевищує 1000, ваш розв'язок отримає вердикт `Output isn't correct: Too many resources used`.

Структура машини фіксується перед викликом `find_structure`. Градер є **детермінованим** у тому сенсі, що якщо ви запустите його двічі, і обидва запуски виконають однакову послідовність операцій, виклики `collect` повернуть ті самі масиви.

Обмеження

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Підзадачі

Підзадача	Бали	Додаткові обмеження
1	5	Корінь має рівно M дітей.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Без додаткових обмежень.

У підзадачах 3 та 4 ваш бал залежить від значення C наступним чином.

Підзадача 3 (25 балів)

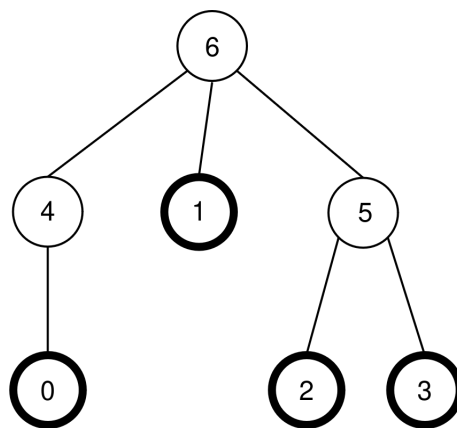
Обмеження	Бали
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Підзадача 4 (60 балів)

Обмеження	Бали
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Приклад

Розглянемо ту саму машину, що й в описі завдання, тобто $N = 7$, $M = 4$ та $P = [4, 6, 5, 5, 6, 6]$. Машину знову показано на наступному рисунку:



Градер робить наступний запуск:

```
find_structure(4)
```

Процедура може виконувати таку послідовність викликів:

1. **insert(0, 0)** : листок 0 вільний, тому кулька зі значенням 0 розміщена на листку 0. Вершина $P[0] = 4$ вільна, тому кулька переміщується до вершини 4. Вершина $P[4] = 6$ вільна, тому кулька переміщується до вершини 6. Оскільки вершина 6 є коренем, рух зупиняється. Виклик повертає `true`.
2. **insert(3, 20)** : листок 3 вільний, тому кулька зі значенням 20 розміщена на листку 3. Вершина $P[3] = 5$ вільна, тому кулька переміщується до вершини 5. Оскільки $P[5] = 6$ зайнята, рух зупиняється. Виклик повертає `true`.
3. **insert(1, 10)** : листок 1 вільний, тому куля зі значенням 10 розміщується на листку 1. Оскільки $P[1] = 6$ зайнята, куля залишається на вершині 1. Виклик повертає `true`.
4. **insert(2, 30)** : листок 2 вільний, тому куля зі значенням 30 розміщується на листку 2. Оскільки $P[2] = 5$ зайнята, куля залишається на вершині 2. Виклик повертає `true`.

5. `insert(1, 25)` : листок 1 зайнятий, тому куля не розміщена на листку 1. Виклик повертає `false`.
6. `insert(0, 20)` : листок 0 вільний, тому кулька зі значенням 20 розміщена на листку 0. Вершина $P[0] = 4$ вільна, тому кулька переміщується до вершини 4. Оскільки $P[4] = 6$ зайнята, рух зупиняється. Виклик повертає `true`.

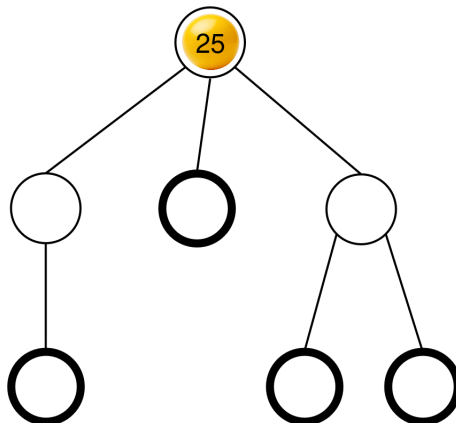
Отримана конфігурація кульок вже була відображена в описі завдання.

Далі процедура може викликати:

```
collect()
```

Виконання цього виклику вже було пояснено в описі завдання, тому цей виклик повертає $S = [0, 10, 20, 30, 20]$. Після цього машина знову порожня.

Далі процедура може викликати `insert(2, 25)`. Листок 2 вільний, тому кулька зі значенням 25 розміщується на листку 0. Потім ця кулька переміщується до вершини 5, а потім до вершини 6, де вона зупиняється. Виклик повертає `true`. Результуюча конфігурація кульок показана на наступному рисунку.



Зрештою, процедура може викликати `collect()`, яка повертає $S = [25]$ та спорожняє машину.

Існує два правильних батьківських масиви, які може повернути `find_structure` : $R = P = [4, 6, 5, 5, 6, 6]$ (що відповідає мітці $L = [0, 1, 2, 3, 4, 5, 6]$), та $R = [5, 6, 4, 4, 6, 6]$ (що відповідає мітці $L = [0, 1, 2, 3, 5, 4, 6]$). У цьому прикладі $K = 2$ та $B = 30$, тому $C = 32$.

Приклад градера

формат вхідних даних:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

формат вихідних даних:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Тут Q це довжина масиву R повернутого функцією `find_structure`.